

Team: 1	Student ID	Student name
	e2401834	Karyna Misnik

Questions

1. What is the main purpose of DevOps?

DevOps main purpose is to shorten the systems development cycle and deliver high-quality software continuously.

2. What are the main stages of a CI/CD pipeline?

- Source/Code : a developer writes code, pushes it to a version-control system (eg. Git);
- Build: the application is built and dependencies are prepared;
- Test: automated tests are executed to find bugs;
- Package: the application is prepared as a deployable artifact;
- Deploy: the application is deployed to an environment (eg. production);
- Monitor: the deployed application is monitored for errors;

3. Which DevOps tools can be used for source control, CI/CD, containerization, infrastructure, configuration management, and monitoring?

DevOps area	Purpose	Example tools
Source control	Store and manage code and track changes	Git, GitHub, GitLab, Bitbucket
CI/CD	Automate building, testing, and deployment	Jenkins, GitLab CI/CD, GitHub Actions, CircleCI
Containerization	Package applications with their dependencies	Docker, Podman
Infrastructure	Create and manage infrastructure using code	Terraform, AWS CloudFormation
Configuration management	Automate configuration and setup of servers/systems	Ansible, Puppet, Chef
Monitoring	Monitor system health, performance, and errors	Prometheus, Grafana, ELK Stack

4. How can Git support effective collaboration in DevOps?

Git supports DevOps collaboration by allowing developers to work on separate branches, review changes through pull/merge requests, and discuss changes using comments. It also provides version history and integrates with project-management and CI/CD tools.

5. What is Infrastructure as Code (IaC), and what are its benefits?

Infrastructure as Code (IaC) is a DevOps practice of managing and providing infrastructure using code instead of manual processes. It saves time and makes deployments repeatable, consistent, and easier to automate.

6. What is configuration management, and how can Ansible support it?

Configuration management is the process of automatically managing and maintaining the configuration of servers and software. Ansible supports it by using code/playbooks to automate tasks such as installing software, changing settings, and configuring multiple servers consistently.

7. What are containers, and why are they useful in DevOps?

Containers package an application together with its dependencies and required environment. They are useful in DevOps because the same container image can be deployed consistently across different environments, reducing differences between development, testing, and production.

8. What is container orchestration, and what role does Kubernetes play?

Container orchestration is the automated management of many containers, including deployment, scaling, networking, and recovery. Kubernetes is a platform that automates and manages containerized applications across multiple machines.

9. What are monitoring and observability, and which tools support them?

Monitoring is continuously checking a system's health, performance, and availability. Observability helps understand why a system behaves a certain way using data such as logs, metrics, and traces. Tools include Prometheus, Grafana, and ELK Stack.

10. What is DevSecOps, and how does it improve software security?

DevSecOps integrates security practices into the development and DevOps process instead of checking security only at the end. It improves security by finding and fixing vulnerabilities earlier, often using automated security testing and monitoring.

11. Why is automated testing important in DevOps?

Automated testing is important in DevOps because tests can run quickly and repeatedly whenever code changes. It helps find bugs early, maintain software quality, and safely automate CI/CD pipelines.

12. What is configuration drift, and how can it be reduced?

Configuration drift happens when systems that should have the same configuration become different over time due to manual or untracked changes. It can be reduced by using Infrastructure as Code, configuration management tools such as Ansible, and automated checks.

13. Why are small and frequent deployments considered a DevOps best practice?

Small and frequent deployments reduce the risk of large changes and make problems easier to identify and fix. They also provide faster feedback and allow users to receive improvements sooner.

14. What are the main steps for responding to a failed deployment?

Main steps: Detect the failure → investigate the cause → rollback or fix the deployment → test the solution → redeploy → monitor the system. Then document the problem and improve the process to prevent it from happening again.

15. How can Git, Jenkins, Docker, Kubernetes, Terraform, Ansible, Prometheus, and Grafana work together?

They can form a DevOps toolchain.

Git → Jenkins → Docker → Kubernetes

↑

Terraform → Infrastructure

↓

Prometheus → Grafana

For example: a developer pushes code to Git → Jenkins builds and tests it → Docker creates an image → Kubernetes deploys it → Prometheus monitors it → Grafana displays the metrics.